5강. React Hook(상태 관리)

React

리엑트 Hook – useState()

- 음료 추가 – 배열 상태 관리

음료 추가

음료 추가 컴포넌트입니다.

현재 음료: 쥬스, 콜라, 커피

음료 추가

리엑트 Hook – useState()

- **음료 추가 – 배열 상태 관리**

```
import { useState } from 'react';

const AddDrink = () => {
  const [drinkName, setDrinkName] = useState(['쥬스', '콜라']);

  const addDrink = () => {
    // 기존의 drinkName 배열에 '커피'를 추가하여 새로운 배열을 생성
    setDrinkName([...drinkName, '커피']);
  };

  return (
    <div>
      <h2>음료 추가</h2>
      <p>음료 추가 컴포넌트입니다.</p>
      <p>현재 음료: {drinkName.join(', ')}</p>
      <button onClick={addDrink}>음료 추가</button>
    </div>
  );
}
```

리엑트 Hook – useState()

- 음료 추가 – 배열 상태 관리

음료 추가

음료 추가 컴포넌트입니다.

음료 이름을 입력하세요 | 음료 추가

- 딸기쥬스
- 카페라떼
- 녹차

리엑트 Hook – useState()

- **음료 추가 – 배열 상태 관리**

```
const AddDrink2 = () => {
  // 음료 이름을 저장하는 상태 변수
  const [drinkName, setDrinkName] = useState([]);
  // 입력 필드의 상태를 관리하기 위한 상태 변수
  const [input, setInput] = useState('');

  // 음료 추가 함수
  const handleAddDrink = () => {
    const newDrink = input.trim();
    if (newDrink === '') {
      alert('음료 이름을 입력해주세요.');
      return;
    }
    // 새로운 음료를 drinkName 상태에 추가
    setDrinkName([...drinkName, newDrink]);
    setInput(''); // 입력 필드 초기화
  };
```

리엑트 Hook – useState()

- **음료 추가 – 배열 상태 관리**

```
return (
  <div>
    <h2>음료 추가</h2>
    <p>음료 추가 컴포넌트입니다.</p>
    <input
      type="text"
      placeholder="음료 이름을 입력하세요"
      value={input}
      onChange={(e) => setInput(e.target.value)}
    />{" "}
    <button onClick={handleAddDrink}>음료 추가</button>

    {/* <p>현재 음료: {drinkName.join(', ')}</p> */}
    {/* map()으로 출력 */}
    <ul>
      {drinkName.map((drink, index) => (
        <li key={index}>{drink}</li>
      ))}
    </ul>
  </div>
);
```

리엑트 Hook – useState()

- **컴포넌트 분리하고 props 사용하기**

AddDrink2.jsx

```
return (
  <div>
    <h2>음료 추가</h2>
    <p>음료 추가 컴포넌트입니다.</p>
    <input
      type="text"
      placeholder="음료 이름을 입력하세요"
      value={input}
      onChange={(e) => setInput(e.target.value)}
    />{" "}
    <button onClick={handleAddDrink}>음료 추가</button>

    {/* DrinkList 컴포넌트에 drinkName 상태를 전달 */}
    <DrinkList drinks={drinkName} />
  </div>
);
```

리엑트 Hook – useState()

- 컴포넌트 분리하고 props 사용하기

DrinkList.jsx

```
const DrinkList = ({ drinks }) => {
  return (
    <ul>
      {drinks.map((drink, index) => (
        <li key={index}>{drink}</li>
      ))}
    </ul>
  );
}

export default DrinkList;
```

할 일 관리(Todo-app)

- 할 일 관리

할 일 관리(Todo-app)

- 할 일 관리

```
function App() {
  const [todos, setTodos] = useState([
    { id: 1, text: '운동 하기', completed: false },
    { id: 2, text: '영화 보기', completed: false },
  ])
  const [inputValue, setInputValue] = useState('')

  console.log(todos.length); //2

  // 입력값 변경 핸들러
  const handleInputChange = (e) => {
    console.log(e.target.value);
    setInputValue(e.target.value)
  }
```

할 일 관리(Todo-app)

- 할 일 관리

```
// 할 일 추가
const handleAddTodo = () => {
  if (inputValue.trim() !== '') {
    // 새로운 할 일 객체 생성
    const newTodo = {
      id: todos.length + 1,
      text: inputValue,
      completed: false,
    }
    // 새로운 할 일을 기존 할 일 목록에 추가
    setTodos([...todos, newTodo])
    setInputValue('') // 입력 필드 초기화
  }
}
```

할 일 관리(Todo-app)

- 할 일 관리

```
// 할 일 완료 체크
const handleToggleComplete = (id) => {
  /* 해당 id를 가진 할 일의 completed 상태를 토글
     todos 배열을 순회하면서 id가 일치하는 할 일의 completed 값을 반전시킴
     id가 일치하지 않는 할 일은 그대로 유지*/
  setTodos(
    todos.map((todo) =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo
    )
  )
}

// 할 일 삭제
const handleDeleteTodo = (id) => {
  /* 해당 id를 가진 할 일을 삭제
     todos 배열을 필터링하여 id가 일치하지 않는 할 일만 남김*/
  setTodos(todos.filter((todo) => todo.id !== id))
}
```

할 일 관리(Todo-app)

- 할 일 관리

```
return (
  <>
    <div className='container'>
      <h2>Todo List</h2>
      <input
        type="text"
        value={inputValue}
        onChange={handleInputChange}
        placeholder="할 일을 입력하세요"
      />
      <button onClick={handleAddTodo}>추가</button>
```

할 일 관리(Todo-app)

- 할 일 관리

```
        {/* 할 일 목록 */}
        <ul className='todo-list'>
          {todos.map((todo) => (
            <li key={todo.id} className={todo.completed ? 'completed' : ''}>
              <input
                type="checkbox"
                checked={todo.completed}
                onChange={() => handleToggleComplete(todo.id)}
              />
              {todo.text}
              <button onClick={() => handleDeleteTodo(todo.id)}>삭제</button>
            </li>
          ))}
        </ul>
      </div>
    </>
  )
}
```

할 일 관리(Todo-app)

- 할 일 관리

```
body {
  background-color: #f1fff0;
  margin: 0;
  padding: 20px;
}
.container{
  max-width: 400px;
  margin: 0 auto;
  background-color: white;
  padding: 20px;
  border-radius: 8px;
  text-align: center;
}
.container h2{
  color: #00f;
}
```

App.css

할 일 관리(Todo-app)

- 할 일 관리

```
.todo-list{
  list-style: none;
  padding: 0;
  margin-top: 30px;
}
.todo-list li{
  margin-bottom: 10px;
}
.todo-list li input[type="checkbox"]{
  margin-right: 10px;
}
.completed{
  text-decoration: line-through;
  color: gray;
}
button{
  margin-left: 10px;
}
```

App.css

할 일 관리(Todo-app)

- **TodoList 컴포넌트 분리하기**

```
return (
  <>
    <div className='container'>
      <h2>Todo List</h2>
      <input
        type="text"
        value={inputValue}
        onChange={handleInputChange}
        placeholder="할 일을 입력하세요"
      />
      <button onClick={handleAddTodo}>추가</button>

      {/* 할 일 목록 */}
      <TodoList
        todos={todos}
        onToggleComplete={handleToggleComplete}
        onDeleteTodo={handleDeleteTodo}
      />
    </div>
```

할 일 관리(Todo-app)

- **TodoList 컴포넌트 만들기**

```
function TodoList({ todos, onToggleComplete, onDeleteTodo }) {
  return (
    <ul className='todo-list'>
      {todos.map((todo) => (
        <li key={todo.id} className={todo.completed ? 'completed' : ''}>
          <input
            type="checkbox"
            checked={todo.completed}
            onChange={() => onToggleComplete(todo.id)}
          />
          {todo.text}
          <button onClick={() => onDeleteTodo(todo.id)}>삭제</button>
        </li>
      ))}
    </ul>
  )
}
```

Hook – useEffect()

- **useEffect()**

useEffect 훅은 함수형 컴포넌트에서 **사이드 이펙트(side effect)**를 처리하기 위해 사용하는 핵심 훅입니다.

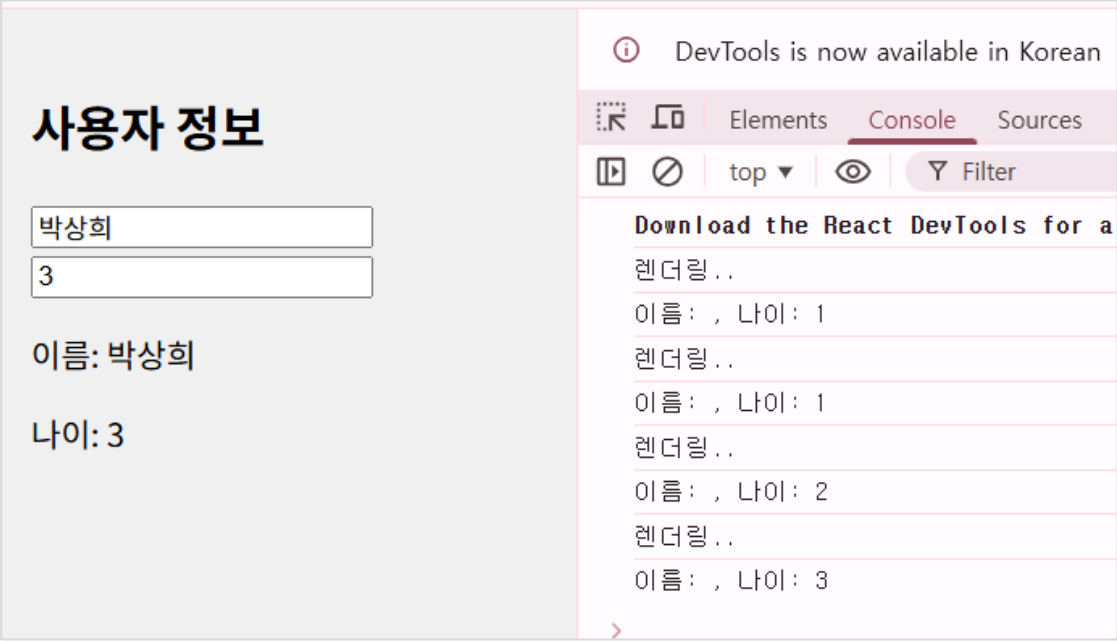
- **데이터 가져오기 (API 호출)**
- **이벤트 등록**
- **타이머 설정**
- **DOM 직접 조작**

```
useEffect(() => {
  // 실행할 코드
}, [의존성]);
```

```
useEffect(() => {
  console.log("count 변경됨");
}, [count]);
```

Hook – useEffect()

- **useEffect()**



Hook – useEffect()

- **useEffect()**

```
import { useEffect, useState } from 'react';

const User = () => {
  const [name, setName] = useState("");
  const [age, setAge] = useState(1);

  // [] -> 처음 렌더링될 때만 실행
  // [age] -> age가 변경될 때마다 실행
  useEffect(() => {
    console.log("렌더링..");
    console.log(`이름: ${name}, 나이: ${age}`);
  }, [age]);

  // 이름을 변경하는 이벤트 핸들러 함수
  const onChangeName = (e) => {
    setName(e.target.value);
  }

  // 나이를 변경하는 이벤트 핸들러 함수
  const onChangeAge = (e) => {
    setAge(Number(e.target.value));
  }
```

Hook – useEffect()

- **useEffect()**

```
return (
  <div>
    <h2>사용자 정보</h2>
    <input
      type="text"
      value={name}
      onChange={onChangeName}
    />
    <br />
    <input
      type="number"
      value={age}
      onChange={onChangeAge}
    />
    <p>이름: {name}</p>
    <p>나이: {age}</p>
  </div>
)
```

Hook – useEffect()

- **useEffect()**

디지털 시계

현재 시간: 오전 11:51:11

Hook – useEffect()

- **useEffect()**

```
import React, { useState, useEffect } from "react";

export default function Clock() {
  // time 상태와 setTime 함수를 useState 훅을 사용하여 정의
  const [time, setTime] = useState(new Date().toLocaleTimeString());

  // useEffect 훅을 사용하여 컴포넌트가 처음 렌더링될 때 타이머를 설정
  useEffect(() => {
    const timer = setInterval(() => {
      setTime(new Date().toLocaleTimeString());
    }, 1000);
    console.log("렌더링..");

    // 컴포넌트가 언마운트될 때 타이머를 정리
    return () => clearInterval(timer);
  }, []); // 빈 배열을 전달하여 컴포넌트가 처음 렌더링될 때만 실행
```

Hook – useEffect()

- **useEffect()**

```
  return (
    <div>
      <h2>디지털 시계</h2>
      <h3>현재 시간: {time}</h3>
    </div>
  );
}
```

실습 문제

- **문제**

useEffect를 사용하여 컴포넌트가 처음 실행될 때 콘솔 출력.

```
페이지 로딩...
```

실습 문제

- **문제**

  입력값을 관리하는 회원가입 폼을 만드세요.

```
이름
이메일
비밀번호
```